# SmartMega Cores: Android CarPower


1、概要 2

2、CarPower整体架构 2

3、CarPower启动流程时序 3

4、定义CarPower框架中的电源状态机 4

5、电源状态机 12

**5.1 Android原生 & 项目中的电源状态机 12**

**5.2 电源状态机迁移(状态机跳转)流程时序 13**

**5.3 休眠流程 14**

6、对上层应用的接口 15

**6.1 Android Car原生接口 15**

**6.2 Android App 如何监听电源状态机的变化 16**

**6.3 由Android侧发起电源状态机切换相关接口 17**

7、CarPower的镁佳平台化实施 18

**7.1 背景 18**

**7.2 平台实现 18**

**7.3 项目定制 20**

**7.3.1 如何新增电源状态机 20**

**7.3.2 如何给项目配置mega_power_policy.xml 25**

8、参考资料 28

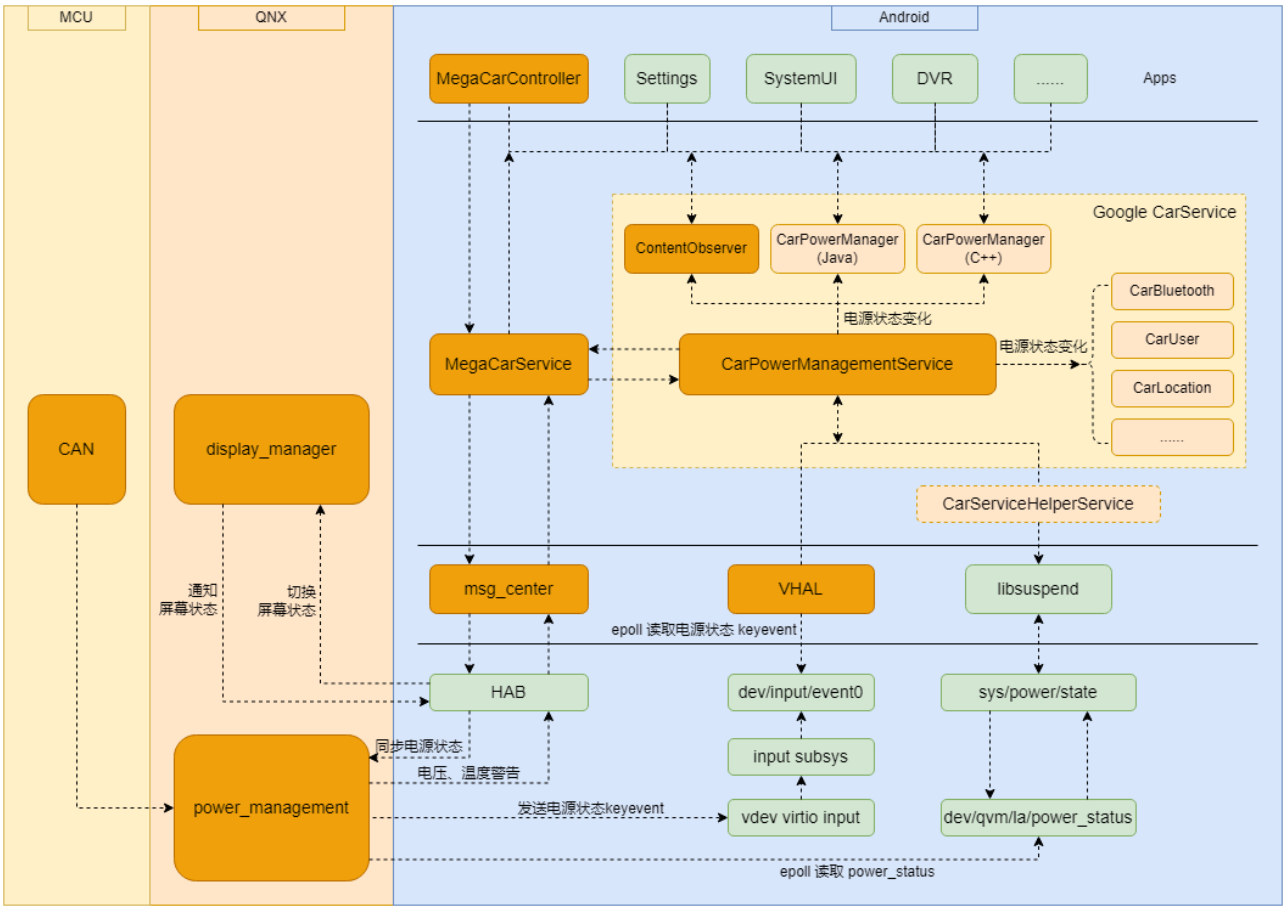
9、目前问题和后续工作 29


## 1、概要

**电源管理分MCU侧和SoC侧，SoC侧包含QNX侧和Android侧。在电源管理整体架构中，以MCU为主控负责整机的电源状态管理维护并将电源状态向上分发，SoC侧由QNX与MCU对接，接收电源状态机的变化做出响应，并将电源状态机继续向上分发给Android侧CarPowerManagementService，CPMS针对不同电源状态机的切换做对应处理并继续将模式切换通知给Android侧监听电源状态的App。**

**本文主要介绍Android侧电源管理在基于Android原生(高通修改后的)基础框架的设计，以及镁佳在基础框架上的扩展设计。对于8155平台QNX侧的电源管理设计，文档见《SA8155 (HQX) 电源管理 - 设计和实现》。**

## 2、CarPower整体架构




CarPower 基于Google原生架构下做了部分改动：

1. 通讯链路：在Vehicle基础上增加了KeyEvents按键监听，QNX层通过发送Input按键通知Vehicle，Vehicle将接收到的Input按键转换为对应的电源状态切换事件通知到CPMS；
2. 电源状态机：基于Google原生电源状态机CarPower框架，在VHAL中增加了待机模式(standby)、远程启动模式(remote_boot)、屏保模式(screen_saver)、临时工作模式(temporary_work)、高压异常(over_voltage)、低压异常(low_voltage)、ABANDONED模式(abandoned)；
3. 电源状态机切换通知：无同步需求的场景下，新增了ContentObserver方式通知Android App电源状态机状态的变化，App层无需接入Android Car SDK。

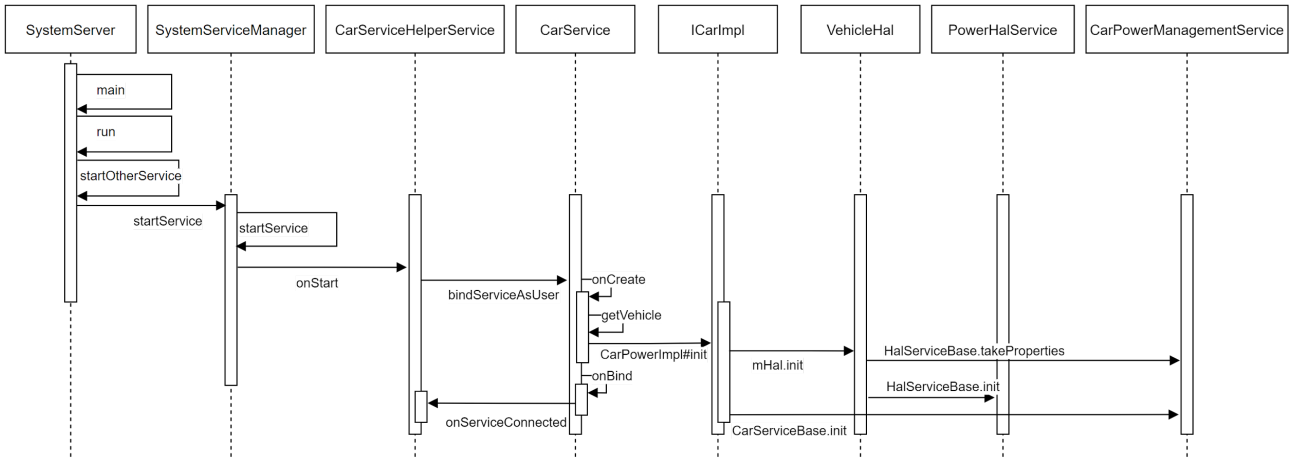
Google原生设计下，Android层的各进程被动接收电源状态机的变化并做出相应的响应处理、但是不支持从Android端发起电源状态机状态的切换，无法满足车辆各场景下的电源状态机切换需求。因此在原生框架下做了以下新增：

1. 屏幕状态切换：非休眠场景下，Android处于活跃状态，屏幕亮灭状态通过发送Topic通知QNX的display_manager进行主动切换。屏幕状态切换执行完成后通过Topic通知回Android；
2. 电源状态切换：在特定场景下，可以由Android层通过发送Topic的方式通知QNX层主动将当前电源状态机切换到期望的目标模式下。QNX收到电源状态机切换请求后，在满足切换条件的时候，通过发送Input按键通知到Vehicle，向上传递电源状态机的切换事件；

Android Automotive 12原生的构架图和本章开头的构架图几乎是一一对应的，其中我们在Android 11上修改时并没有实现的是用于动态调整Power Policy的Policy Daemon部分。

## 3、CarPower启动流程时序

我们并没有对Android Automotive的CarPower流程进行修改。这里简单介绍其启动流程，方便大家迅速了解CarPower是如何启动并和VHAL层建立连接的。




Google Automotive在Android原有框架基础上增加了新的SystemService（即：CarServiceHelperService）用来启动com.android.car进程，并绑定到CarService上同com.android.car交互。本图只介绍com.android.car中CarPower相关的启动流程。其启动流程如上图所示：

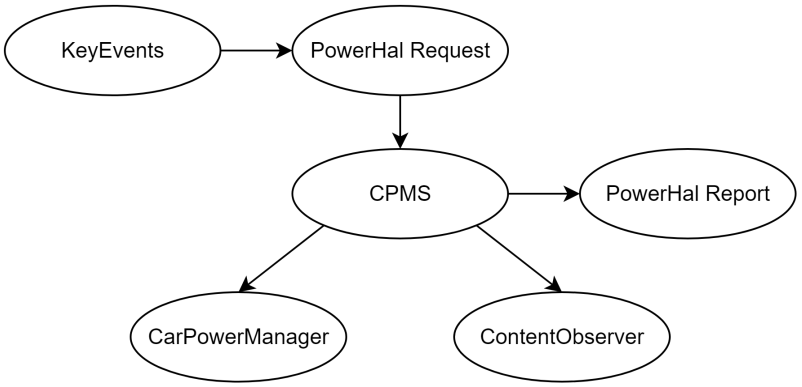
1. Android系统SystemServer启动后，通过调用SystemServiceManager#startService来启动CarServiceHelperService；
2. CarServiceHelperService启动后通过bindService的方式启动com.android.car进程Service组件Android原生的CarService（注：不是镁佳用于实现车辆属性访问的XxxCarService。Android原生的CarService集成在com.android.car中，涉及到CarBluetooth、CarLocation、CarDog等模块需要使用CarPowerManager相关接口）；
3. CarService启动时初始化ICar.Stub的实现类ICarImpl，并在onBind生命周期将ICarImpl返回给CarServiceHelperService；
4. ICarImpl在init过程中将包括CarPowerManagementService在内的接口CarServiceBase实现类都实例化并通过调用init函数执行初始化流程；
5. 同时ICarImpl实例化VehicleHal调用其init函数进行VehicleHal的初始化过程
6. 在VehicleHal的初始化过程中，VehicleHal将所有HalBaseService的实现类集中实例化并调用init()进行初始化，其中包括PowerHalService。从而将上层的CPMS和PowerHal建立联系。

## 4、定义CarPower框架中的电源状态机

Android原生电源状态机定义分为

- VehicleHal侧Request状态/Report状态
- CarPowerManagementService侧的电源状态机
- CarPowerManager侧的电源状态机

VehicleHal侧上报过来的Request状态会映射为CarPowerManagementService侧的电源状态机，用于上报给CarPowerManagementService进行对应的电源状态机状态切换。
CarPowerManagementService在进行电源状态机状态切换时，会将当前的电源状态机映射为VehicleHal侧Report状态用来向下通报给PowerHal，同时也会将当前的电源状态机映射为CarPowerManaer侧的电源状态机用于通知到App层的监听Listener。各个模块之间的电源状态机State值的映射关系如图：




镁佳在Android原生之外新增加的模式是从项目中逐渐抽象出来的（如：下面这些表格中标注了一些模式在特定项目中会使用到）。Android侧在代码上是平台化的，所有的项目编译出来的CarPower都有这些KEY；而实际项目中Android侧的电源状态由QNX侧上报过来的，不一定会全部使用到每一个KEY（如果QNX侧不上报给Android该状态，则Android侧不会处理该状态）。如果QNX上报给Android特定状态，该项目的Android侧电源管理机制会根据mega_power_policy.xml的配置，来决定Android侧是否处理接收到的电源状态。

1、VehicleHal侧Request的电源状态机定义如下(绿色为Android原生，蓝色为镁佳新增，下同)：

| 模式 | 取值 | 描述 |
|---|---|---|
| ON | 0 | 正常工作模式 |
| SHUTDOWN_PREPARE | 1 | ShutDown准备阶段 |
| CANCEL_SHUTDOWN | 2 | 取消ShutDown |
| FINISHED | 3 | ShutDown complete |
| STANDBY | 4 | 待机模式 |
| SCREEN_SAVER | 5 | 屏保模式(奇瑞) |
| TEMPORARY_WORKING | 6 | 临时工作模式(奇瑞) |
| REMOTE_BOOT | 7 | 远程启动模式(奇瑞) |
| OVER_VOLTAGE | 8 | 高压异常模式(奇瑞) |
| LOW_VOLTAGE | 9 | 低压异常模式(奇瑞) |
| ABANDONED | 0xa | Abandon模式(岚图) |
| PLAYING_BOOTANIMATION | 0xb | 开机动画模式 |
| OTA | 0xc | OTA升级模式 |

| | | |
|---|---|---|
| FACTORY_MODE | 0xD | Factory Mode (奇瑞T28车辆模式) |
| TRANSPORT_MODE | 0xE | Transport Mode (奇瑞T28车辆模式) |
| DYNO_MODE | 0xF | Dyno Mode (奇瑞T28车辆模式) |
| CRASH_MODE | 0x10 | Crash Mode (奇瑞T28车辆模式) |
| FACTORY_PAUSED_MODE | 0x11 | Factory Paused Mode (奇瑞T28车辆模式) |
| TRANSPORT_PAUSED_MODE | 0x12 | Transport Paused Mode (奇瑞T28车辆模式) |

VehicleHal侧Request的电源状态机根据监听到的QNX发送KeyEvent转换过来，QNX在电源状态变化时会发送对应的KeyEvent。目前定义的KeyEvent如下：

| 定义 | Virtio Key | 取值 | 对应的Request模式 |
|---|---|---|---|
| PM_WORKMODE_KEY_EVENT | KEY_F2 | 60 | ON |
| PM_STANDBY_KEY_EVENT | KEY_F3 | 61 | STANDBY |
| PM_STRMODE_KEY_EVENT | KEY_F4 | 62 | SHUTDOWN_PREPARE (STR休眠流程) |
| PM_SCREENSAVER_KEY_EVENT | KEY_F5 | 63 | SCREEN_SAVER |
| PM_TEMPORARY_WORK_KEY_EVENT | KEY_F6 | 64 | TEMPORARY_WORKING |
| PM_REMOTESTART_KEY_EVENT | KEY_F7 | 65 | REMOTE_BOOT |
| PM_OTAMODE_KEY_EVENT | KEY_F8 | 66 | OTA |
| PM_POWERON_KEY_EVENT | KEY_F9 | 67 | 弃用 |
| PM_OVER_VOLTAGE_KEY_EVENT | KEY_F10 | 68 | OVER_VOLTAGE |
| PM_LOW_VOLTAGE_KEY_EVENT | KEY_F11 | 87 | LOW_VOLTAGE |
| PM_ABANDONED_KEY_EVENT | KEY_F12 | 88 | ABANDONED |
| PM_SHUTDOWN_ONLY_KEY_EVENT | KEY_EJECTCD | 161 | SHUTDOWN_PREPARE (关机流程) |
| PM_CANCEL_SUSPEND_KEY_EVENT | KEY_TUNER | 386 (0x182) | CANCEL_SHUTDOWN (关机、休眠中断) |
| PM_BOOTANIMATION_KEY_EVENT | KEY_RIGHTMETA | 126 | PLAYING_BOOTANIMATION |
| PM_FACTORY_MODE | KEY_A | | FACTORY_MODE |
| PM_TRANSPORT_MODE | KEY_B | | TRANSPORT_MODE |
| PM_DYNO_MODE | KEY_C | | DYNO_MODE |
| PM_CRASH_MODE | KEY_D | | CRASH_MODE |
| PM_FACTORY_PAUSED_MODE | KEY_E | | FACTORY_PAUSED_MODE |
| PM_TRANSPORT_PAUSED_MODE | KEY_F | | TRANSPORT_PAUSED_MODE |

2、CarPowerManagementService侧的电源状态机定义如下：

| 模式 | 取值 | 描述 |
| --- | --- | --- |
| WAIT_FOR_VHAL | 0 | 等待Vhal |
| ON | 1 | 正常工作模式 |
| SHUTDOWN_PREPARE | 2 | ShutDown准备阶段 |
| WAIT_FOR_FINISH | 3 | ShutDown准备完成，等待进入Finish (Suspend) |
| SUSPEND | 4 | Suspend模式 (Finish) |
| SIMULATE_SLEEP | 5 | 模拟休眠模式 |
| STANDBY | 6 | 待机模式 |
| SCREEN_SAVER | 7 | 屏保模式(奇瑞) |
| TEMPORARY_WORKING | 8 | 临时工作模式(奇瑞) |
| REMOTE_BOOT | 9 | 远程启动模式(奇瑞) |
| OVER_VOLTAGE | 10 | 高压异常模式(奇瑞) |
| LOW_VOLTAGE | 11 | 低压异常模式(奇瑞) |
| ABANDONED | 12 | abandon模式(岚图) |
| BOOT_ANIMATION | 13 | 开机动画模式 |
| OTA | 14 | OTA升级模式 |
| FACTORY_MODE | 15 | Factory Mode (奇瑞T28车辆模式) |
| FACTORY_PAUSED_MODE | 16 | Factory Paused Mode (奇瑞T28车辆模式) |
| TRANSPORT_MODE | 17 | Transport Mode (奇瑞T28车辆模式) |
| TRANSPORT_PAUSED_MODE | 18 | Transport Paused Mode (奇瑞T28车辆模式) |
| CRASH_MODE | 19 | Crash Mode (奇瑞T28车辆模式) |
| DYNO_MODE | 20 | Dyno Mode (奇瑞T28车辆模式) |

3、VehicleHal侧Report的电源状态机定义如下：

| 模式 | 取值 | 描述 |
| --- | --- | --- |
| WAIT_FOR_VHAL | 0x1 | 等待VHAL |
| DEEP_SLEEP_ENTRY | 0x2 | 进入深度休眠 |
| DEEP_SLEEP_EXIT | 0x3 | 退出深度休眠(唤醒) |
| SHUTDOWN_POSTPONE | 0x4 | ShutDown延迟 |

| | | |
|---|---|---|
| SHUTDOWN_START | 0x5 | 开始ShutDown |
| ON | 0x6 | 工作模式 |
| SHUTDOWN_PREPARE | 0x7 | ShutDown准备阶段 |
| SHUTDOWN_CANCELLED | 0x8 | 取消ShutDown |
| OVER_VOLTAGE | 0x9 | 高压异常 |
| NORMAL | 0xa | 正常模式(从异常模式退出的场景) |

4、CarPowerManager侧的电源状态机定义如下：
(SA8295/SA8255平台Android 13/14有变化，具体见 [SA8295/SA8255平台] CarPower接口 )

| 模式 | 取值 | 描述 |
|---|---|---|
| INVALID | 0 | 无效状态 |
| WAIT_FOR_VHAL | 1 | 等待Vhal |
| SUSPEND_ENTER | 2 | 进入Suspend |
| SUSPEND_EXIT | 3 | 退出Suspend |
| SHUTDOWN_ENTER | 5 | 进入ShutDown |
| ON | 6 | 正常工作模式(通常我们说的ACTIVE) |
| SHUTDOWN_PREPARE | 7 | ShutDown准备阶段 |
| SHUTDOWN_CANCELLED | 8 | 取消ShutDown |
| STANDBY | 9 | 待机模式 |
| SCREEN_SAVER | 10 | 屏保模式(奇瑞) |
| TEMPORARY_WORKING | 11 | 临时工作模式(奇瑞) |
| REMOTE_BOOT | 12 | 远程启动模式(奇瑞) |
| OVER_VOLTAGE | 13 | 高压异常模式(奇瑞) |
| LOW_VOLTAGE | 14 | 低压异常模式(奇瑞) |
| ABANDONED | 15 | ABANDONED模式(岚图，其表现和STANDBY一致) |
| BOOT_ANIMATION | 16 | 开机动画模式 |
| OTA | 17 | OTA升级模式 |
| FACTORY_MODE | 18 | Factory Mode (奇瑞T28车辆模式) |
| FACTORY_PAUSED_MODE | 19 | Factory Paused Mode (奇瑞T28车辆模式) |
| TRANSPORT_MODE | 20 | Transport Mode (奇瑞T28车辆模式) |

| | | |
|---|---|---|
| TRANSPORT_PAUSED_MODE | 21 | Transport Paused Mode (奇瑞T28车辆模式) |
| CRASH_MODE | 22 | Crash Mode (奇瑞T28车辆模式) |
| DYNO_MODE | 23 | Dyno Mode (奇瑞T28车辆模式) |

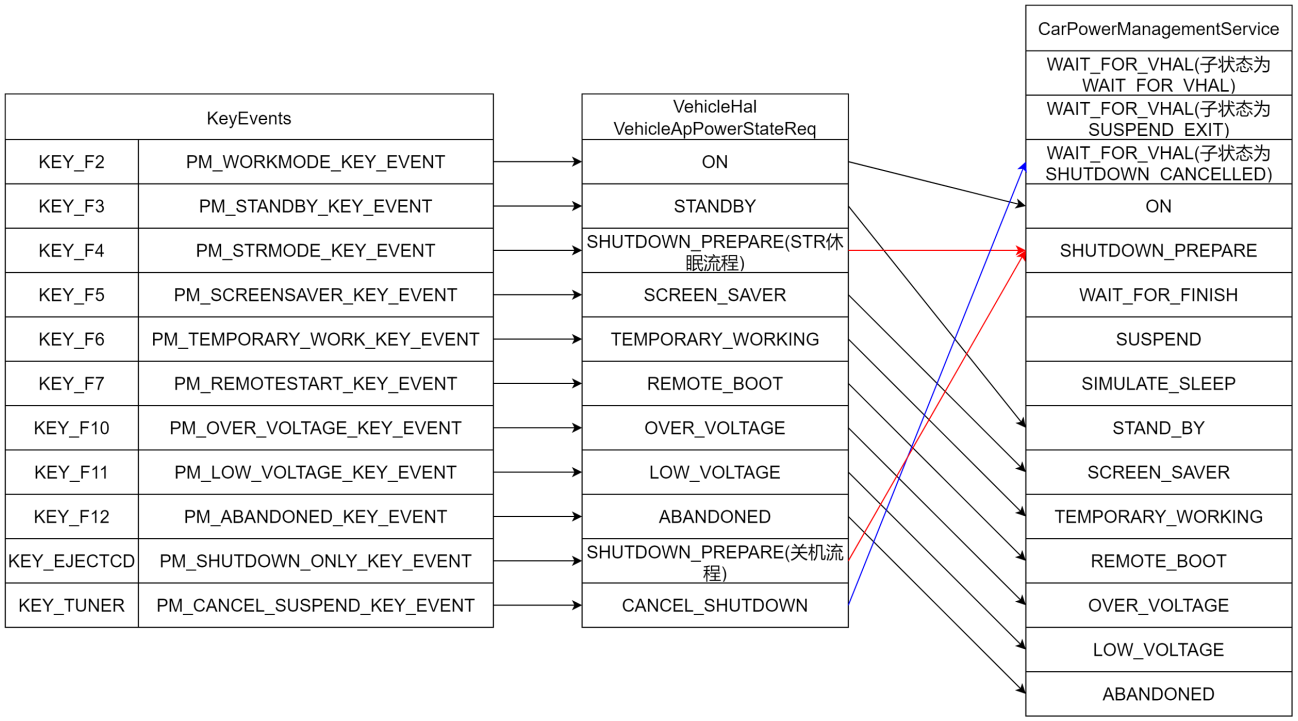
5、通过ContentObserver提供给App监听，在MegaSettings中定义的模式定义如下：
(App若使用ContentObserver监听电源状态机的变化，则集成
mega.nexus.framework.static.source.jar，对应的模式code在该jar中的MegaSettins中定义)

(SA8295平台Android 13有变化，具体见 [SA8295/SA8255平台] CarPower接口)

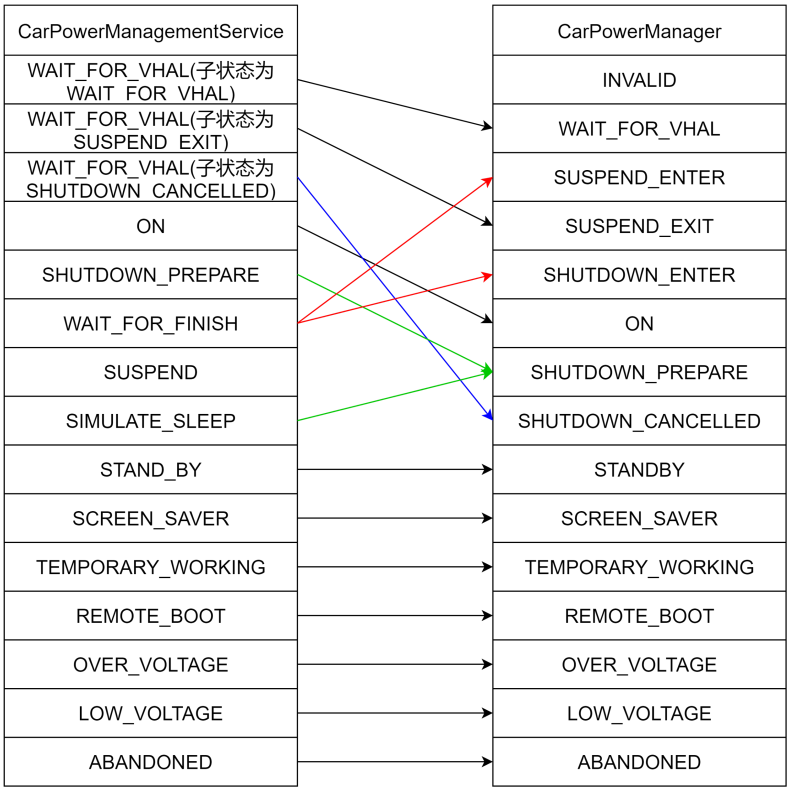
| 模式 | 取值 | 描述 |
|---|---|---|
| STATUS_WAIT_FOR_VHAL | 0 | 等待Vhal |
| STATUS_ON | 1 | 工作模式 |
| STATUS_SHUTDOWN_PREPARE | 2 | ShutDown准备阶段 |
| STATUS_WAIT_FOR_FINISH | 3 | ShutDown准备完成，等待进入Finish |
| STATUS_SUSPEND | 4 | Suspend模式 |
| STATUS_SIMULATE_SLEEP | 5 | 模拟Sleep |
| STATUS_STAND_BY | 6 | 待机模式 |
| STATUS_SCREEN_SAVER | 7 | 屏保模式(奇瑞) |
| STATUS_TEMPORARY_WORKING | 8 | 临时工作模式(奇瑞) |
| STATUS_REMOTE_BOOT | 9 | 远程启动模式(奇瑞) |
| STATUS_OVER_VOLTAGE | 10 | 高压异常模式(奇瑞) |
| STATUS_LOW_VOLTAGE | 11 | 低压异常模式(奇瑞) |
| ABANDONED | 12 | ABANDONED模式(岚图) |
| STATUS_BOOT_ANIMATION | 13 | 开机动画模式 |
| OTA | 14 | OTA升级模式 |
| STATUS_FACTORY | 15 | Factory Mode (奇瑞T28车辆模式) |
| STATUS_FACTORY_PAUSED | 16 | Factory Paused Mode (奇瑞T28车辆模式) |
| STATUS_TRANSPORT | 17 | Transport Mode (奇瑞T28车辆模式) |
| STATUS_TRANSPORT_PAUSED | 18 | Transport Paused Mode (奇瑞T28车辆模式) |
| STATUS_CRASH | 19 | Crash Mode (奇瑞T28车辆模式) |
| STATUS_DYNO | 20 | Dyno Mode (奇瑞T28车辆模式) |

6、各个模块之间电源状态机的映射关系如下图：

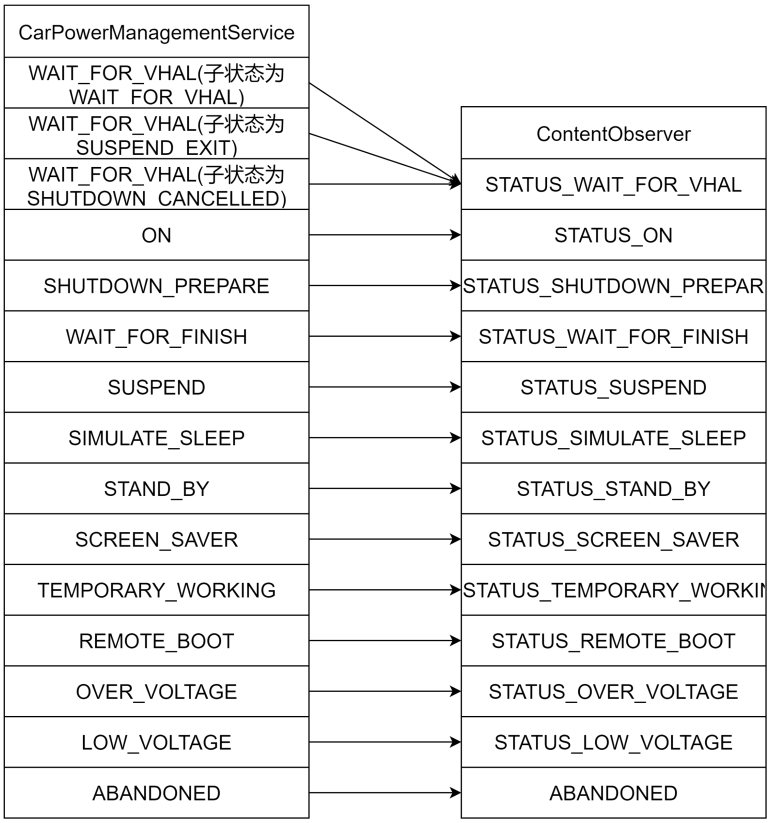
## KeyEvents > VehicleHal request > CarPowerManagementService



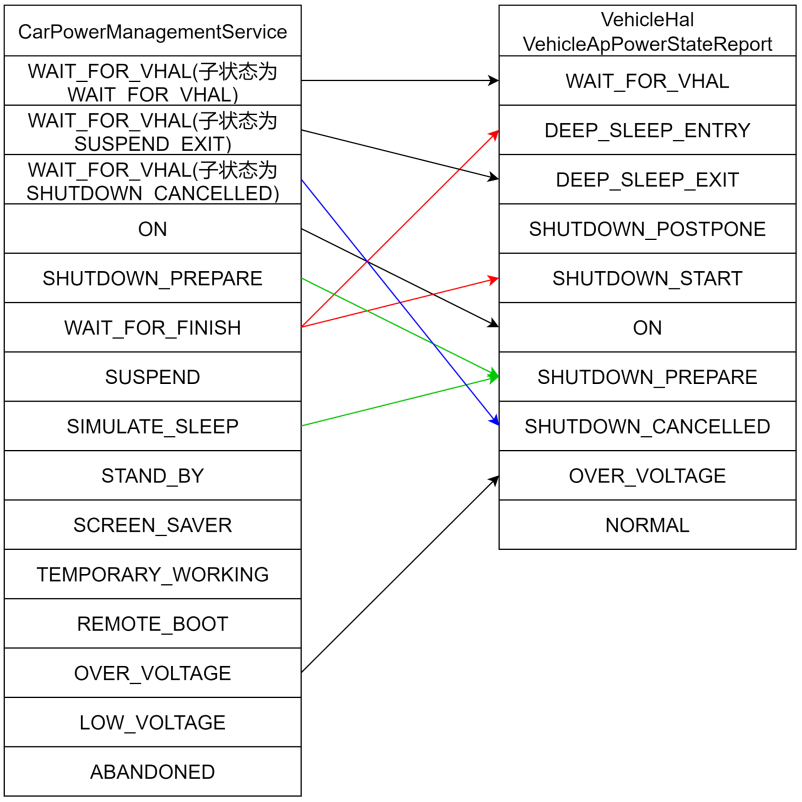

## CarPowerManagementService > CarPowerManager




## CarPowerManagementService > ContentObserver



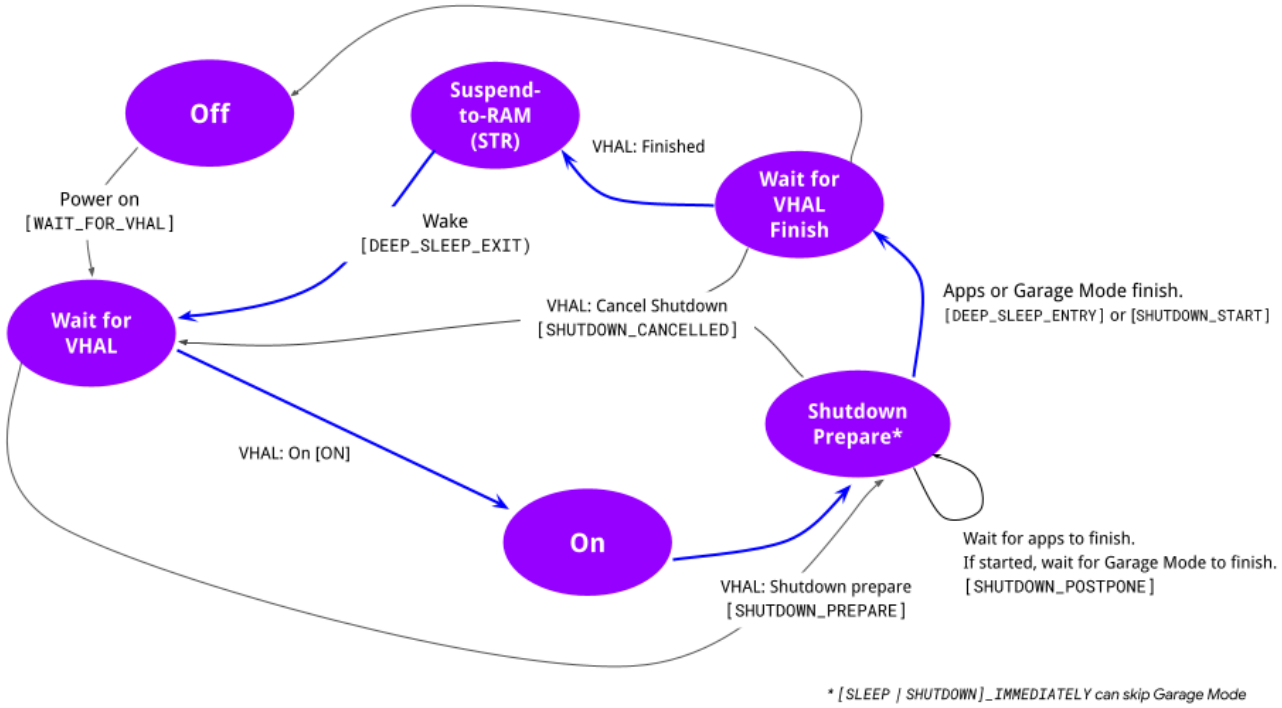

## CarPowerManagementService > VehicleHal report



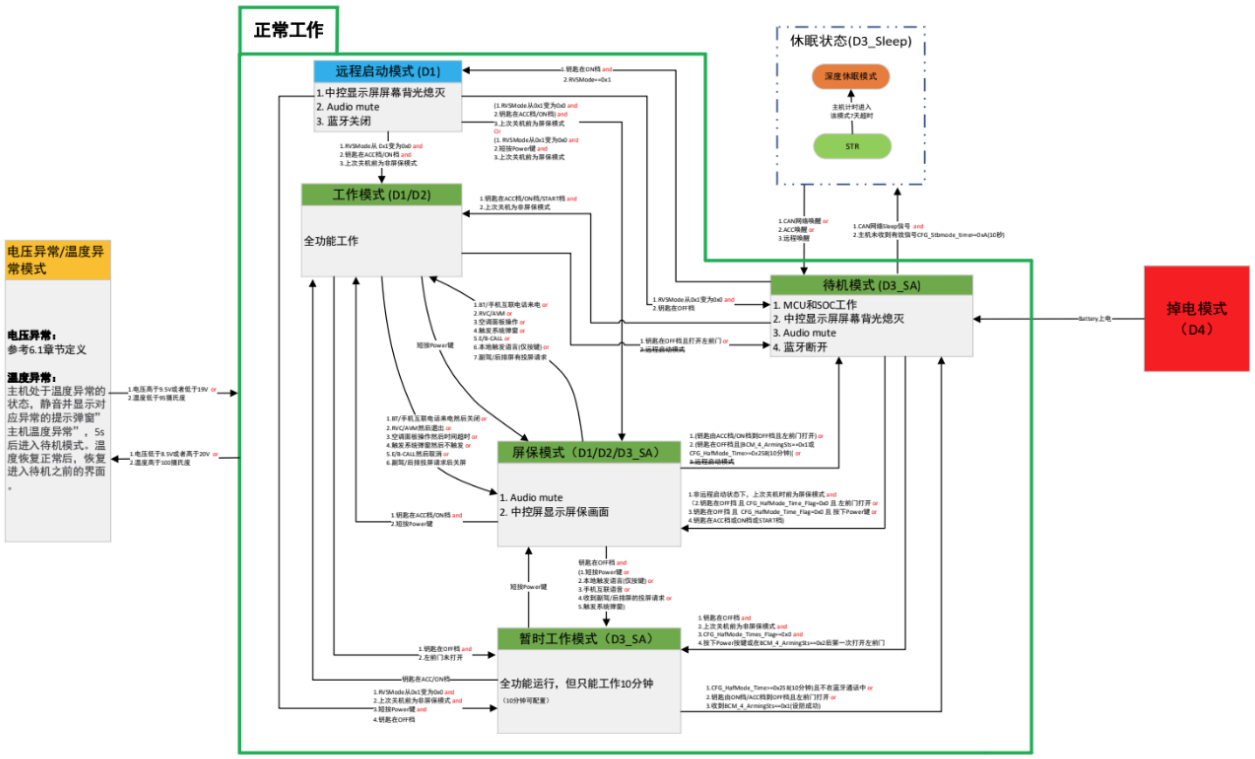
# 5、电源状态机

## 5.1 Android原生 & 项目中的电源状态机

**Android原生电源状态机图：**

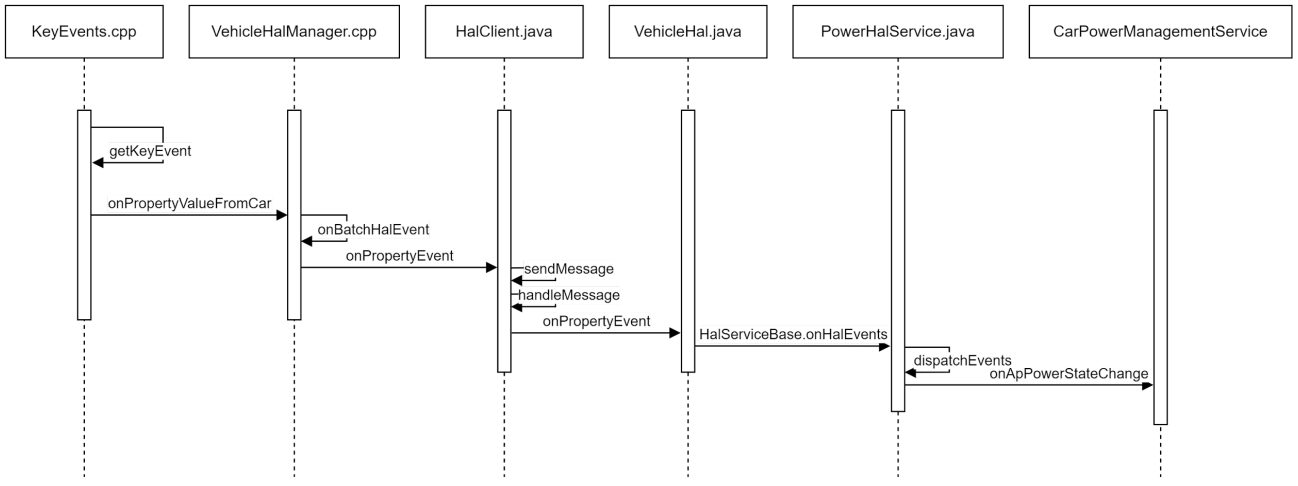



**镁佳基于奇瑞项目的电源管理状态机图如下(相关链接)：**




**对于项目中的电源管理状态机要求，镁佳在Android侧基本上是在Google原生框架上进行了扩展来实现；STR休眠流程按照Google框架来执行。状态机的定义以及需要修改的代码见章节7.3.1介绍；而policy.xml主要工作是定义不同状态机下的component行为。**

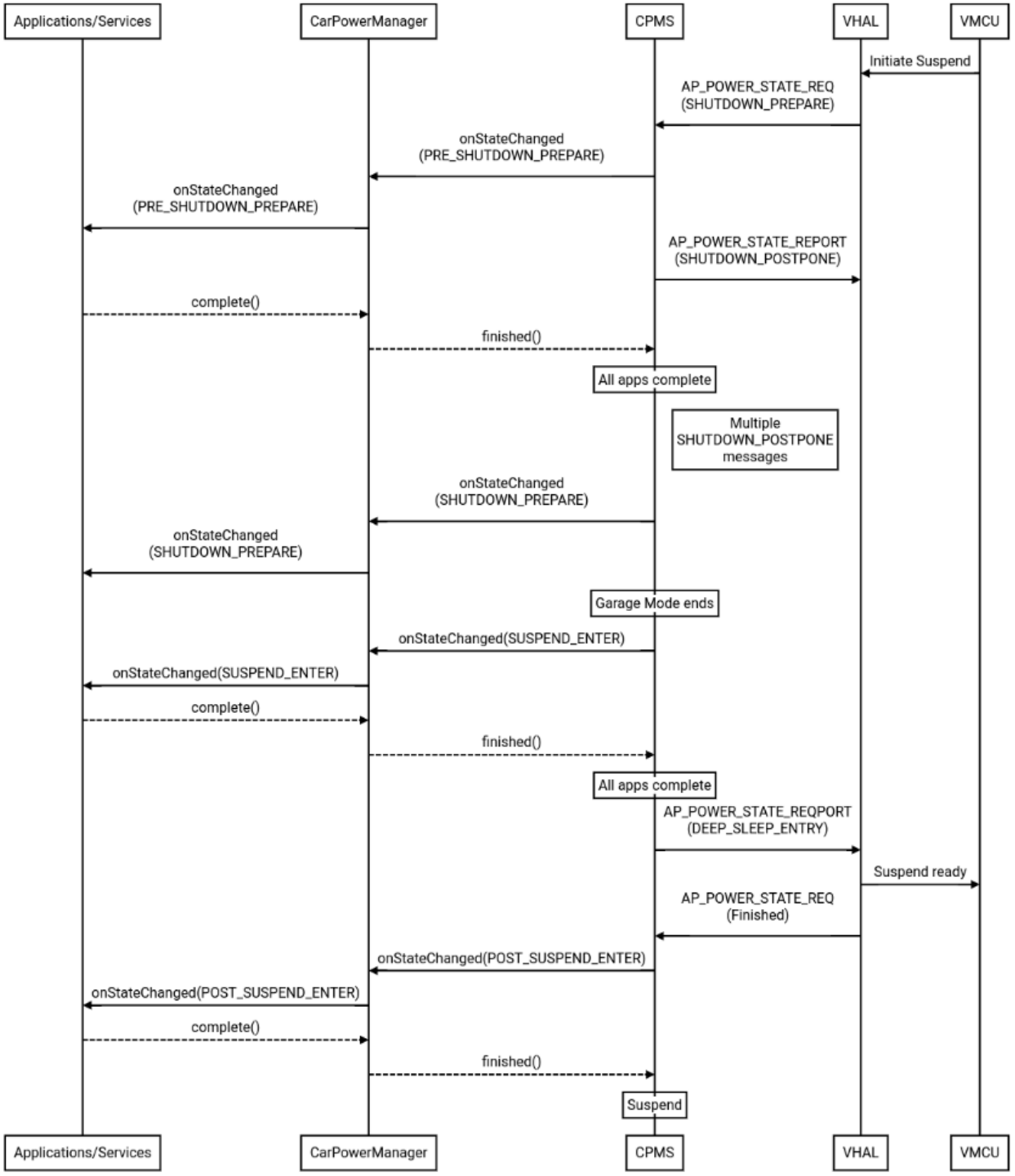
## 5.2 电源状态机迁移(状态机跳转)流程时序




CarPower

## 5.3 休眠流程

**休眠流程基本上遵守android原生流程，部分细节做了改动，不再赘述。流程参考如下Android原生流程图：（Google官方链接 电源管理-消息序列-进入深度休眠）**




# 6、对上层应用的接口

**SA8295平台Android 13有变化，具体见 [SA8295平台] CarPower接口**

## 6.1 Android Car原生接口

**连接Android Car以获取CarPowerManager实例**

```
CarPowerManager mCarPowerManager = null;
Car.createCar(mContext, mMainHandler, Car.CAR_WAIT_TIMEOUT_WAIT_FOREVER, new
Car.CarServiceLifecycleListener() {
    @Override
    public void onLifecycleChanged(Car car, boolean ready) {
        if (ready) {
          (mCarPowerManager == null) {
                mCarPowerManager = (CarPowerManager) car.getCarManager(
                        Car.POWER_SERVICE);
                if (mCarPowerManager != null) {
                    mCarPowerManager.clearListener();
                    mCarPowerManager.setListener(carPowerStateListener);
                }
            }
        } else {
            mCarPowerManager = null;
        }
    }
});
```

**获取当前电源状态机 (CarPowerManager#CarPowerStateListener 中定义的状态机)**

```
public int getPowerState() {
    try {
        return mService.getPowerState();
    } catch (RemoteException e) {
        return handleRemoteExceptionFromCarService(e, CarPowerStateListener.INVALID);
    }
}
```

**注册电源状态机非同步监听listener**

```
public void setListener(CarPowerStateListener listener) {
    synchronized (mLock) {
        if (mListener != null || mListenerWithCompletion != null) {
            throw new IllegalStateException("Listener must be cleared first");
        }
        // Update listener
        mListener = listener;
        setServiceForListenerLocked(false);
    }
}
```

**注册电源状态机同步监听listener**

```
public void setListenerWithCompletion(CarPowerStateListenerWithCompletion listener) {
    synchronized (mLock) {
        if (mListener != null || mListenerWithCompletion != null) {
            throw new IllegalStateException("Listener must be cleared first");
        }
```

```
        // Update listener
        mListenerWithCompletion = listener;
        setServiceForListenerLocked(true);
    }
}
```

**移除电源状态机监听listener**

```
public void clearListener() {
    ICarPowerStateListener listenerToService;
    synchronized (mLock) {
        listenerToService = mListenerToService;
        mListenerToService = null;
        mListener = null;
        mListenerWithCompletion = null;
        cleanupFutureLocked();
    }

    if (listenerToService == null) {
        Log.w(TAG, "unregisterListener: listener was not registered");
        return;
    }

    try {
        mService.unregisterListener(listenerToService);
    } catch (RemoteException e) {
        handleRemoteExceptionFromCarService(e);
    }
}
```

## 6.2 Android App 如何监听电源状态机的变化

**经过镁佳修改的Android侧CarPower在Java层对外提供三种接口：**

1. **集成android.car.jar (android版本有差异，不同版本集成不同的jar)，使用Android原生CarPowerManager提供的两种注册监听方式：**
   **(需要在AndroidManifest.xml中注册权限 android.car.permission.CAR_POWER，该权限需要平台签名)**
   a. **CarPowerManager#setListener(CarPowerStateListener listener)**
      **该方式注册监听电源状态机时，App收到电源状态机的变化后、只需要关注自己的业务处理即可；**
   b. **CarPowerManager#setListenerWithCompletion(CarPowerStateListenerWithCompletion listener)**
      **该方式注册监听电源状态机时，调用在onStateChanged回调中。App收到电源状态机变化、处理完自己的业务后，需要调用CompletableFuture#complete同步通知CarPowerManagementService。注:同步listener只能system uid的app使用**

   **Demo参考示例 AndroidCarPowerDemo.java**
2. **集成mega.nexus.framework.static.source.jar，镁佳增加的通过ContentObserver方式来监听电源状态机的变化：对电源状态机变化不敏感的非粘性需求App可使用ContentObserver监听**

的方式来监听电源状态机的变化，无需集成Android原生android.car.jar。只需要使用Android原生接口registerContentObserver注册监听即可，uri name为“mega_android_power_status”。

Android 11 版本Demo参考示例 MegaCarPowerDemo.java

Android 13 版本Demo参考示例 MegaCarPowerDemo13.java

3. Mega增加的通过注册监听电源状态机Topic方式来监听电源状态机的变化（这种方法脱离了Android的CarPower体系、使得系统中出现不受电源状态机管理的行为，不建议Android侧App使用）。

注：如果Android侧系统服务中要监听电源状态机，可使用ContentObserver的方式来监听，URI、电源状态机都在frameworks/base/core/java/android/provider/Settings.java中有定义

### 6.3 由Android侧发起电源状态机切换相关接口

CarPowerManager增加接口 requestToSwitchCarPowerState 提供给 Android app调用，用于主动发起电源状态机的切换。

```
/**
 * request CarPower to switch power state
 * @param targetState target state to be switched. see {@link SwitchStateEnum}
 * @param reason the reason why to switch power state
 * @hide
 */
public void requestToSwitchCarPowerState(@NonNull @SwitchStateEnum int targetState,
                                         @NonNull String reason) {
    try {
        String pkg = getContext().getPackageName();
        mService.syncCarPowerState(targetState, 0, pkg + "_" + reason);
    } catch (RemoteException e) {
        handleRemoteExceptionFromCarService(e);
    }
}
```

## 7、CarPower的镁佳平台化实施

### 7.1 背景

不同项目不同车厂对于电源状态机的定义不尽相同，并且在相同的电源状态机下对于蓝牙、WiFi、屏幕等外设的处理也不尽相同。Android原生的电源管理下仅对蓝牙、WiFi、语音助手、屏幕进行了处理，对于麦克风、USB、Audio等模块均没有做处理。如：奇瑞项目中客户需求在不同的电源状态机下对麦克风、Audio等均做对应处理；而长安项目上相同的电源状态机下部分模块未做处理要求。

镁佳之前的实现方式为在packages/services/Car/service/res/values/configs.xml下增加配置项，不同项目通过overlay的方式来定义不同配置项的具体值。在代码中，我们使用hard code的方式读取配置项的值来对外设模块进行处理。该方式有两大弊端：

1. 代码臃肿，Android原生的CPMS被改的面目全非；

2. 配置项过多，并且没有和电源状态机有任何关联，当后期项目越来越多的时候，配置项可能就无法满足不同项目不同电源状态机下对外设的具体处理行为。

为了解决以上两大弊端，减小镁佳/项目逻辑对Android原生代码的侵入性，并且能够较全面的覆盖到不同外设在不同电源状态机下的具体处理。我们参考Android 12 上对于外设处理的配置逻辑，对其进行改造，以适应当前Android 11上镁佳结合不同项目的需求。

## 7.2 平台实现

在Android 12上，Google将不同电源状态机下的外设处理行为做成了一套power_policy的xml，在policy中，定义了不同的PolicyGroup，在PolicyGroup下定义不同的policy对应不同的电源状态机，每个policy下定义了外设模块的具体行为(enable、disable、untouched)。在此基础上，我们将Android 12这套机制进行了删减改造，去掉了noDefaultPolicy、systemPolicyOverrides等我们不需要的镁佳定义，并且删减了允许动态调整policy的机制，在保留尽量少的代码和功能逻辑下实现能够适应mega项目的需求。

在Android 11 上我们目前实现的policy策略仅有defaultPolicy，在defaultPolicy下我们支持不同的policy下对具体的外设模块component进行行为定义，该policy下的其他需要做统一处理的外设模块使用otherComponents定义具体的行为。policy配置文件名为 mega_power_policy.xml：

- 文件路径：device/mega/项目名/config/mega_power_policy.xml
- 编译刷机后在设备中的路径为：system/etc/config/mega_power_policy.xml

在Android端的平台化修改包括：

1. Android上层和底层/QNX的通讯方式：在SA8155平台上，电源状态机的变化通过QNX写入Input设备节点通知Android。Android侧则通过epoll读取Input设备节点获取QNX上报过来的电源状态；
   注：代码路径参考 hardware/interfaces/automotive/vehicle/2.0/default/KeyEvents.cpp
2. 在VHAL和CarPower服务中新增了电源状态机，调整了各个状态机之间的跳转规则；
   如何新增电源状态机请参考章节7.3.1。
   状态机之间的跳转规则限制参考代码
   packages/services/Car/service/src/com/android/car/CarPowerManagementService#needPowerStateChangeLocked
3. 提供不同状态机下的外设组件 (component) 的行为：Android 11上只有背光、Touch、蓝牙、WiFi，并且背光、Touch的处理和项目实际实施有差异。Android 12上只提供了policy的框架、但并没有各个component的实现，我们在以下路径中提供了自己的实现。
   注：各component的实现代码在
   packages/services/Car/service/src/com/android/car/mega/component
4. 修改了休眠唤醒后的原生Android流程：原生的唤醒流程是通知HAL后直接切换到ON模式，我们改为向QNX侧同步(向QNX侧电源管理模块下发同步请求Topic)，QNX会根据当前的整车状态(包括车辆信号)来判定当前需要上报给Android的电源状态机。

## 7.3 项目定制

开发一个特定车型项目的电源管理时，需要进行以下工作：

- 若有新增电源状态机，则需要从VHAL层到上层CarServic定义电源状态机KEY，以提供整个链路的电源状态机映射，并提供给三方监听APP上报电源状态机切换，具体定义方法和需要修改的模块参考本章节后续的例子；(注：这部分工作建议通过向IFS-Cores团队提出需求的方式来完成)

- 在整车层面上定义Power Policy，即：mega_power_policy.xml（路径：device/mega/xxx项目名/config/mega_power_policy.xml），具体定义方法参考本章中后序的例子；
- 需要响应电源状态变化事件的应用（App），按照第6章的方法实现其响应逻辑。

### 7.3.1 如何新增电源状态机

目前已经在CarPower框架中拓展实现的电源状态机基本能满足绝大多数项目需要。当一个新的项目启动时，应首先了解现在的电源状态机定义，尽量选择已经实现了的电源状态机、或者向IFS-Cores团队提出需求对现有电源状态机进行适当调整。

例如新增一个电源状态机为STANDBY待机模式，需做如下工作：
1）与CBS部门负责QNX侧Power的同事沟通定义好STANDBY待机模式下QNX侧Power写入Input设备节点的KEY值为多少。例如STANDBY待机模式时，QNX侧Power写入Input设备节点的是KEY值为KEY_F3，value取值为61；Android侧需要在VHAL层automotive 模块修改以下代码文件：
hardware/interfaces/automotive/vehicle/2.0/default/KeyEvents.h
增加枚举值 PM_STANDBY_KEY_EVENT = KEY_F3,

```
typedef enum {
    PM_WORKMODE_KEY_EVENT = KEY_F2,
    PM_STANDBY_KEY_EVENT = KEY_F3,   // 定义STANDBY的KEY
    PM_STRMODE_KEY_EVENT = KEY_F4,
    PM_SCREENSAVER_KEY_EVENT = KEY_F5,
    PM_TEMPORARY_WORK_KEY_EVENT = KEY_F6,
    PM_REMOTESTART_KEY_EVENT = KEY_F7,
    PM_SLEEPMODE_KEY_EVENT = KEY_F8,
    PM_POWERON_KEY_EVENT = KEY_F9,
    PM_OVER_VOLTAGE_KEY_EVENT = KEY_F10,
    PM_LOW_VOLTAGE_KEY_EVENT = KEY_F11,
    PM_ABANDONED_KEY_EVENT = KEY_F12,
} pm_key_event_t;
```

2）在Android VHAL层automotive模块定义KEY_F3需要映射的VHAL侧VehicleApPowerStateReq上报的模式。修改以下代码文件：hardware/interfaces/automotive/vehicle/2.0/types.hal，增加VehicleApPowerStateReq枚举值STANDBY=4：

```
enum VehicleApPowerStateReq : int32_t {
    ON = 0,
    SHUTDOWN_PREPARE = 1,
    CANCEL_SHUTDOWN = 2,
    FINISHED = 3,
    STANDBY = 4,
    SCREEN_SAVER = 5,
    TEMPORARY_WORKING = 6,
    REMOTE_BOOT = 7,
    OVER_VOLTAGE = 8,
    LOW_VOLTAGE = 9,
    ABANDONED = 0xa,
};
```

同时修改KeyEvents.cpp，将epoll读取Input设备KEY_F3，枚举值PM_STANDBY_KEY_EVENT映射为 VehicleApPowerStateReq::STANDBY。修改以下代码文件：
hardware/interfaces/automotive/vehicle/2.0/default/KeyEvents.cpp

```
case PM_STANDBY_KEY_EVENT:
    server->onPropertyValueFromCar(
    *CreateApPowerStateReq(VehicleApPowerStateReq::STANDBY, 0), true);
    break
```

3）在Android CarPower侧PowerHalService中添加VehicleApPowerStateReq枚举值STANDBY的String字符(这里主要是便于log分析排查问题)。修改以下代码文件：
packages/services/Car/service/src/com/android/car/hal/PowerHalService.java

```
private static String powerStateReqName(int state) {
    String baseName;
    switch(state) {
        case VehicleApPowerStateReq.ON:
            baseName = "ON";
            break;
        case VehicleApPowerStateReq.SHUTDOWN_PREPARE:
            baseName = "SHUTDOWN_PREPARE";
            break;
        case VehicleApPowerStateReq.CANCEL_SHUTDOWN:
            baseName = "CANCEL_SHUTDOWN";
            break;
        case VehicleApPowerStateReq.FINISHED:
            baseName = "FINISHED";
            break;
        case VehicleApPowerStateReq.STANDBY:
            baseName = "STANDBY";
            break;
        case VehicleApPowerStateReq.SCREEN_SAVER:
            baseName = "SCREEN_SAVER";
            break;
        case VehicleApPowerStateReq.TEMPORARY_WORKING:
            baseName = "TEMPORARY_WORKING";
            break;
        case VehicleApPowerStateReq.REMOTE_BOOT:
            baseName = "REMOTE_BOOT";
            break;
        case VehicleApPowerStateReq.OVER_VOLTAGE:
            baseName = "OVER_VOLTAGE";
            break;
        case VehicleApPowerStateReq.LOW_VOLTAGE:
            baseName = "LOW_VOLTAGE";
            break;
        case VehicleApPowerStateReq.ABANDONED:
            baseName = "ABANDONED";
            break;
        default:
            baseName = "<unknown>";
```

```
            break;
    }
    return baseName + "(" + state + ")";
}
```

4）在CarPowerManagementService中增加CpmsState电源状态机定义STAND_BY，将VehicleApPowerStateReq.STANDBY 映射为CpmsState.STAND_BY，并定义电源状态机跳转限制规则。修改如下代码：

packages/services/Car/service/src/com/android/car/CarPowerManagementService.java

```
// ADD STAND_BY
public static final int STAND_BY = 6;
```

```
case VehicleApPowerStateReq.STANDBY:
    this.mCanPostpone = false;
    this.mCanSleep = false;
    this.mCarPowerStateListenerState = cpmsStateToPowerStateListenerState(STAND_BY);
    this.mState = STAND_BY;
    break;
```

电源状态机跳转限制规则修改CarPowerManagementService#needPowerStateChangeLocked

```
case CpmsState.STAND_BY:
case CpmsState.SIMULATE_SLEEP:
case CpmsState.OVER_VOLTAGE:
case CpmsState.LOW_VOLTAGE:
case CpmsState.ABANDONED:
    transitionAllowed = true;
    break;
```

5）增加STANDBY需要向外通知的CarPowerManager侧的电源状态机、ContentObserver侧的电源状态机、VehicleHal侧Report的电源状态机；

CarPowerManager侧电源状态机

a、在CarPowerManager#CarPowerStateListener中的电源状态机定义；

b、在CarPowerManagementService中将CpmsState#mCarPowerStateListenerState映射为CarPowerManager#CarPowerStateListener的定义值；

修改以下文件：

packages/services/Car/car-lib/src/android/car/hardware/power/CarPowerManager.java

```
/**
* Enter standby state. CPMS is switching to STAND_BY state.
* @hide
*/
int STANDBY = 9;
```

packages/services/Car/service/src/com/android/car/CarPowerManagementService.java 以及CarPowerManagementService#cpmsStateToPowerStateListenerState

```
case VehicleApPowerStateReq.STANDBY:
    this.mCanPostpone = false;
```

```
    this.mCanSleep = false;
    this.mCarPowerStateListenerState = cpmsStateToPowerStateListenerState(STAND_BY);
    this.mState = STAND_BY;
    break;
```

```
private static int cpmsStateToPowerStateListenerState(int state) {
    int powerStateListenerState = 0;

    // Set the CarPowerStateListenerState based on current state
    switch (state) {
        case ON:
            powerStateListenerState = CarPowerStateListener.ON;
            break;
        case SHUTDOWN_PREPARE:
            powerStateListenerState = CarPowerStateListener.SHUTDOWN_PREPARE;
            break;
        case SUSPEND:
            powerStateListenerState = CarPowerStateListener.SUSPEND_ENTER;
            break;
        case STAND_BY:
            powerStateListenerState = CarPowerStateListener.STANDBY;
            break;
        case SCREEN_SAVER:
            powerStateListenerState = CarPowerStateListener.SCREEN_SAVER;
            break;
        case TEMPORARY_WORKING:
            powerStateListenerState = CarPowerStateListener.TEMPORARY_WORKING;
            break;
        case OVER_VOLTAGE:
            powerStateListenerState = CarPowerStateListener.OVER_VOLTAGE;
            break;
        case LOW_VOLTAGE:
            powerStateListenerState = CarPowerStateListener.LOW_VOLTAGE;
            break;
        case ABANDONED:
            powerStateListenerState = CarPowerStateListener.ABANDONED;
            break;
        case WAIT_FOR_VHAL:
        case WAIT_FOR_FINISH:
        default:
            // Illegal state for this constructor.  Throw an exception?
            break;
    }
    return powerStateListenerState;
}
```

**ContentObserver侧电源状态机**

**a、在frameworks/base/core/java/android/provider/Settings.java增加STATUS_STAND_BY定义；**

**b、在**

vendor/mega/packages/services/MegaNexus/nexus-frameworks/static/com/mega/nexus/provider/MegaSettings.java中将Settings定义的STATUS_STAND_BY暴露给App使用。

VehicleHal侧发送电源状态机
VehicleHal侧发送电源状态机用于给HAL层监听CPMS向下通知电源状态。修改以下文件，增加枚举值 VehicleApPowerStateReport：hardware/interfaces/automotive/vehicle/2.0/types.hal

6）添加CarPower对新增电源状态机的处理逻辑
在CarPowerManagementService中对新增的电源状态机进行业务处理，主要修改CarPowerManagementService#doHandlePowerStateChange。
当Android侧CarPower接收到qnx上报的电源状态机的变化后，会在doHandlePowerStateChange函数中做对应电源状态机的业务处理。
具体处理的业务包括：

- Android原生平台化模块处理，如wifi、蓝牙的开关处理；
- 触发mega_power_policy机制，根据policy配置对相关component进行开关处理；
- 将当前电源状态机通知给监听端的app；

## 7.3.2 如何给项目配置mega_power_policy.xml

每个电源状态机的具体行为由mega_power_policy.xml定义；项目定制的核心工作是根据客户需求编写这个文件。以长安C385项目为例：

1. 需要声明对外设进行处理的电源状态机，包括：
    - 初始模式 WAIT_FOR_VHAL；
    - 工作模式 ON；
    - 待机模式 STANDBY；
    - 休眠前的第一个状态 SHUTDOWN_PREPARE；
2. 需要进行处理的外设有：Audio、Ethernet、语音助手、Volume、Process进程强杀。

长安项目电源状态机下对外设模块的行为策略的要求如下：

- WAIT_FOR_VHAL 模式：各外设模块均统一不做任何处理，进入该电源状态机之前，外设是什么状态，在该电源状态机下外设就保持什么状态。因此我们定义 otherComponents 的处理行为是 untouched（即：不做任何处理）。
- ON 模式：Audio、Ethernet、语音助手、Volume做Enable处理，其他外设均保持进入该模式前的状态不做任何处理。
- STANDBY 模式： Audio、语音助手做Disable处理，Ethernet、Volume做Enable处理，其他外设保持当前状态不做任何处理。
- SHUTDOWN_PREPARE 模式： Audio、Ethernet、语音助手、Volume、App Process进程均做Disable处理，其他模块保持当前状态不做任何处理。

根据以上电源管理需求，我们在 device/mega/bigsur/config 下新建一个 mega_power_policy.xml 文件。做以下工作：

1. 首先我们定义policyGroups，在policyGroups下我们声明 policyGroup，id定义为 basic_policy_group；

2. 在 basic_policy_group 下我们声明 defaultPolicy，每个defaultPolicy对应长安C385需要做policy处理的电源状态机。defaultPolicy state对应电源状态机，defaultPolicy id定义没有要求，只需要和后面的 policy id 保持一一对应即可；注意 defaultPolicy state 的命名需要和packages/services/Car/service/src/com/android/car/mega/MegaPolicyReader#POWER_STATE_XXX 保持一致；例如长安C385上，我们需要添加 WaitForVHAL、On、StandBy、ShutdownPrepare 四个电源状态机的policy处理，因此需要声明4个defaultPolicy。以StandBy举例，defaultPolicy的声明为：
   `<defaultPolicy state="StandBy" id="basic_policy_03"/>`
3. 声明policies，在policies下我们声明policy，每个policy需要和defaultPolicy一一对应，id和defaultPolicy id一一对应；
4. 在policy下定义component组件的行为：
   - 如果单独对某个component进行行为定义，则使用 component标签。注意：id 的命名规范为 POWER_COMPONENT_XXX，XXX需要和packages/services/Car/service/src/com/android/car/mega/MegaPowerComponentUtil#POWER_COMPONENT_XXX保持一致；行为定义中 on 表示执行 enable操作，off表示执行disable操作，untouched表示保持当前状态不做处理。
     例如需要定义Audio进行enable操作行为，则该component的定义为：
     `<component id="POWER_COMPONENT_AUDIO">on</component>`
   - 除明确需要单独定义component行为的外，其他component可以统一使用otherComponents标签来统一定义行为；
     例如在定义了Audio组件的行为后，其他组件都不需要做任何处理，则可以定义otherComponents为：
     `<otherComponents behavior="untouched"/>`

以长安C385为例，mega_power_policy.xml 的具体配置内容则如下：
(文件路径：device/mega/bigsur/config/mega_power_policy.xml)

```
<?xml version='1.0' encoding='utf-8'?>

<powerPolicy version="1.0">
    <policyGroups>
        <policyGroup id="basic_policy_group">
            <defaultPolicy state="WaitForVHAL" id="basic_policy_01"/>
            <defaultPolicy state="On" id="basic_policy_02"/>
            <defaultPolicy state="StandBy" id="basic_policy_03"/>
            <defaultPolicy state="ShutdownPrepare" id="basic_policy_04"/>
        </policyGroup>
    </policyGroups>

    <policies>
        <!-- state wait for vhal -->
        <policy id="basic_policy_01">
            <otherComponents behavior="untouched"/>
        </policy>
        <!-- state active on -->
        <policy id="basic_policy_02">
            <otherComponents behavior="untouched"/>
            <!-- audio not mute -->
```

```
            <component id="POWER_COMPONENT_AUDIO">on</component>
            <!-- ethernet up -->
            <component id="POWER_COMPONENT_ETHERNET">on</component>
            <!-- voice interaction enable -->
            <component id="POWER_COMPONENT_VOICE_INTERACTION">on</component>
            <!-- volume mount -->
            <component id="POWER_COMPONENT_VOLUME">on</component>
        </policy>
        <!-- state standby -->
        <policy id="basic_policy_03">
            <otherComponents behavior="untouched"/>
            <!--audio mute-->
            <component id="POWER_COMPONENT_AUDIO">off</component>
            <!-- ethernet up -->
            <component id="POWER_COMPONENT_ETHERNET">on</component>
            <!-- voice interaction disable -->
            <component id="POWER_COMPONENT_VOICE_INTERACTION">off</component>
            <!-- volume mount -->
            <component id="POWER_COMPONENT_VOLUME">on</component>
        </policy>
        <!-- state shutdown prepare -->
        <policy id="basic_policy_04">
            <otherComponents behavior="untouched"/>
            <!--audio mute-->
            <component id="POWER_COMPONENT_AUDIO">off</component>
            <!-- ethernet down -->
            <component id="POWER_COMPONENT_ETHERNET">off</component>
            <!-- voice interaction disable -->
            <component id="POWER_COMPONENT_VOICE_INTERACTION">off</component>
            <!-- volume unmount -->
            <component id="POWER_COMPONENT_VOLUME">off</component>
            <!-- force stop packages not in white list -->
            <component id="POWER_COMPONENT_PROCESS">off</component>
        </policy>
    </policies>
</powerPolicy>
```

**注：~~实际上各个项目上还有对于蓝牙、WiFi、屏幕的处理，这三个模块Android原生有对应的处理逻辑和处理方式，并且蓝牙、WiFi模块由CBS同事在BSP平台层维护，因此暂时没有将其做成policy下的component。后期会视情况再看是否要对Android原生代码进行删改，将这部分做成policy下的component进行统一管理。~~**

## 8、参考资料

1、Android原生车载电源管理
2、Chery_8155_Convergence_FRS_Power Moding_V0.9
3、Android 12 原生 Power Policy实现
4、镁佳平台化 Power Policy 实现代码链接
5、代码路径以及仓库：

- 路径：android/packages/services/Car
  仓库：qcom/sa/quic/la/platform/packages/services/Car
- 路径：android/hardware/interfaces/automotive/vehicle/2.0
  仓库：qcom/sa/quic/la/platform/hardware/interfaces
- 路径：android/device/mega/xxxx(项目名)/config/mega_power_policy.xml
  仓库：qcom/sa/quic/la/vendor/mega/device/xxxx(项目名)
- 路径：vendor/qcom/proprietary/vnw/carpower
  仓库：qcom/sa/quic/la/vendor/mega/qcom_android_proprietary

# 9、目前问题和后续工作

**2022/11月剩余问题如下（2022/12均已完成）**

1. **完善Mega定义的电源状态机在Android原生CarPowerManager对外暴露状态的支持**
   → 目前主线 android.car.jar 中已经可以通过原生CarPowerManager获取所有Mega新增定义的电源状态机。jar已经更新到主线，客户端同事可以更新主线代码通过gradle中通过 implementation files("$rootProject.ext.androidCarJar") 引用。
2. **根据各项目在不同模式下以及相同模式下不同外设需求的各个场景，需要针对项目做差异化处理。参考Android 12上的实现方式。(Android 12源码参考)**
   → 针对CarPower平台化的工作，目前已经基本完成。
3. **镁佳8155平台上关机流程整体结构调整，原本方案是qnx直接下发一个长按power key绕过了CarPower体系，直接找到Android的PowerManagerService去做的关机。现在要调整为qnx通知Android侧CarPower，从ShutdownPrepare模式下执行关机流程，复用Android原生方案。**
   → Android侧CarPower改动较小，涉及两处：1）shutdown关机时调用Android PowerMananger#shutdown 需要传入 shutdown reason以通知qnx；2）Android原生ShutdownPrepare模式下等待监听端阻塞的超时时间过长，为15分钟，目前调整到3分钟
4. **要将Android原生CarPower上的蓝牙、wifi、display模块做成component组件利用power_policy来进行管理。**
   → 蓝牙、wifi依旧保持Android原生平台化的处理方式；display在Android原生只有主屏的处理，将其从Android原生中抽离出区分主屏、副屏做成component来根据mega_power_policy.xml配置进行管理。屏幕touch的处理也抽离出做成component来根据mega_power_policy.xml配置来进行管理。
5. **将Android侧发起电源状态机切换集中管理，通过CarPowerManager提供API接口给APP。**
   → 接口已提供，具体使用方法参考章节6.3。
6. **根据奇瑞项目需求变更，在电源状态机变化的时候，对于Audio的处理不能做全局静音，需要根据不同的Audio通道来分别做静音、取消静音的操作。**
   → 增加使用CarAudioManager管理Audio的MegaCarAudioComponent、MegaCarAudioTtsComponent，在奇瑞项目上对于Audio的处理均使用MegaCarAudio的component。其他项目目前仍然使用Android原生的AudioManager通过MegaAudioComponent来操作Audio，如果后续有需求对Audio按通道管理，则可以更换为CarAudio相关的component来配置mega_power_policy.xml。